

3 IRC

Mise en œuvre d'un système à microprocesseur

Quelques rappels

1. Objectifs globaux du module :

- Séance 1 : Prise en main de Microvision. Premiers codes en C
- Séance 2 : Programmation des entrées-sorties.
- Séance 3 : Mise en œuvre d'interruptions
- Séance 4 : Mise en œuvre de Timer
- Séance 5 : Mise en œuvre d'UART
- Séance 6 : Mise en œuvre de périphériques analogiques DAC et ADC
- Séance 7 : Evaluation individuelle - Mise en œuvre d'un périphérique

2. Documentations disponibles sur e-campus :

- Supports de cours
- Initiation à Microvision
- Documentation Microcontrôleur 8051
- Architecture et outils de développement
- Datasheet 8051F020
- 8051 Instruction Set
- Architecture 8051
- Langage C et microcontrôleur 8051

3. Organisation des TP

Les TP se font en binômes, voire en trinômes (selon les contraintes de salle)

4. Répertoire de travail :

Travailler de préférence en local. Projet Microvision sur le disque dur du PC

Placer le projet dans le dossier : **C:\CPE_users\TP_IRC\TPn\noms_étudiants**

5. Evaluation

La note finale de TP est établie en fonction des résultats observés en TP et d'une évaluation individuelle en dernière séance

Mise en œuvre d'un système à microprocesseur
TP6 – ADC et DAC
1. Objectifs de la séance

- Mise en œuvre d'un périphérique ADC
- Mise en œuvre d'un périphérique DAC

2. Rendus de fin de séance

Ces rendus seront faits sur le e-campus.

- Le fichier **Base_TP6.C** contenant l'intégralité des codes réalisés durant le TP renommé impérativement de la manière suivante : **Base_TP6_Nom1_Nom2.C**.

3. Compétences acquises

A l'issue de ce TP, vous devez pouvoir répondre par l'affirmative à ces 2 questions.

- Je sais configurer et utiliser le convertisseur analogique numérique ADC0.
- Je sais configurer et utiliser le convertisseur numérique-Analogique DAC0

Si ne pouvez pas répondre par l'affirmative à ces questions, les objectifs du TP ne sont pas atteints, vous devez donc veiller à combler ces lacunes avant la prochaine séance de TP.

4. Activité 1 – Compréhension des concepts de la conversion A/N

Objectifs : Retrouver quelque caractéristiques d'un CAN

On a branché un capteur de température sur une entrée de convertisseur analogique-numérique

- Le **capteur de température** fournit une tension analogique proportionnelle à la température mesurée, selon la relation : $V_{cap} (V) = (T/100) * 5$. **T** est la température en degrés Celsius ($0^{\circ}C \leq T \leq 100^{\circ}C$).
- Le **convertisseur** analogique-numérique a une résolution de 8 bits, un temps de conversion de 1 milliseconde et une tension de référence de 5V.

Questions :

- **1-Echantillonnage :** Quelle est la fréquence minimale d'échantillonnage à utiliser si le capteur mesure des variations rapides de température avec une fréquence maximale de **10 Hz** ? Par ailleurs, quelle sera la fréquence maximale d'échantillonnage pour ce convertisseur?
- **2-Quantification :** Calculer le **pas de quantification** du CAN, c'est-à-dire la plus petite variation de tension détectable.
- **3-Codage :** Pour une température de **30°C**, déterminer :
 - La tension analogique d'entrée correspondante.
 - La valeur numérique correspondante après quantification.
 - La valeur binaire codée en sortie du CAN (sur 8 bits).
- **4-Si on réduit Vref à 2.5V**, quel sera l'effet sur :
 - Le **pas de quantification** ?
 - La **précision** de la conversion ?
 - La **plage de tension convertissable** ?

5. Activité 2 – Analyse d'un code de configuration et premières conversions A/N

Dans cette partie, vous allez travailler dans le dossier : **C:\CPE_users\TP_IRC\TP6**

Récupérer sur e-campus le fichier ZIP Code_Source_TP6. Ce dossier contient les fichiers suivants :

- Le fichier **Projet_TP6.uvproj** est le fichier de configuration de Microvision adapté à cet exercice.
- Le fichier **Base_TP6.C** est un fichier C, qui contient le programme **main** du projet. Vous le complétez en modifiant le programme **main**, et en ajoutant des fonctions de configurations et d'utilisation.
- Le fichier **Lib_Base.C** contient les fonctions de configuration du microcontrôleur. **Ce fichier ne sera pas modifié.**
- Le fichier **Asm-8051_Lib.asm** contient des fonctions écrites en assembleur et appelables par des programmes C. **Ce fichier ne sera pas modifié.**

Vérifiez que vous obtenez bien le fonctionnement suivant :

- Clignotement régulier de la LED branchée sur P1.6.
- Mise en œuvre d'une liaison série sur l'UART0 configurée à 115200Bd, 8bits de données, pas de parité, 1 stop bit
- Au démarrage du programme le message « **TP6 Archi 3IRC – System Ready** » est envoyé vers le terminal de commande, puis un message « **Mesure xx** » est envoyé chaque seconde.



Les signaux électriques échangés avec la carte 8051F020 sont des niveaux CMOS 0-3,3V au maximum!! Ceci est valable aussi pour les broches véhiculant des signaux analogiques

Vérification de la tension de référence VREF interne

La fonction de configuration `Voltage_Reference_Init()` fournie dans le code permet d'obtenir une tension de référence sur les broches VREFD, VREF0 et VREF1 (valeur identique sur les 3 broches).

- Mesurez la valeur de cette tension de référence obtenue sur les broches VREFD, VREF0 et VREF1.
- Compte tenu de cette tension de référence, déterminez de manière théorique le résultat de la conversion analogique numérique sur le convertisseur ADC0 (12 bits) pour un signal d'entrée d'amplitude 0V, 0,5V, 1V, 1,5V, 2V, 2,5V, 3V. Exprimer cette valeur en décimal et en hexadécimal

Configuration de l'ADC0 – Premiers essais de conversion A/N

Une fonction de configuration `void ADC0_Init(void)` est mise à votre disposition. Analysez son contenu et répondez aux questions suivantes. (La réponse à ces questions est indispensable pour réaliser la suite).

- Sur quelle voie d'entrée (AIN0 à AIN7) la conversion va-elle être réalisée ?
 - Quelle est le gain en tension de l'amplificateur placé en amont du convertisseur ADC0 ?
 - Quel est le temps de conversion de l'ADC0 ?
 - Comment la conversion est-elle déclenchée ?
 - Comment le résultat de conversion est-il stocké dans les registres ADC0H et ADC0L ?
- Testez cette configuration. La procédure de test sera la suivante : toutes les secondes, dans le `while(1)` du `main`, on déclenchera une conversion analogique-numérique du signal appliqué sur l'entrée sélectionnée dans la configuration (on appliquera une tension continue ou très lentement variable de 0 à 3V). Le résultat de la conversion sera transmis (à chaque seconde) au terminal de commande grâce à l'appel de la fonction `printf` : « **Mesure N – ADC0H = XX – ADC0L = YY** » (N variable affichant le numéro de la mesure, XX contenu en hexadécimal du registre ADC0H, YY contenu en hexadécimal du registre ADC0L).
 - Relever les valeurs obtenues pour les tensions d'entrées suivantes : 0V, 0,5V, 1V, 1,5V, 2V, 2,5V, 3V.
 - Vérifier que les valeurs transmises correspondent bien aux valeurs calculées au début de l'activité 2
 - Améliorer votre code en calculant et en affichant la valeur de la tension convertie. Le message à envoyer sera donc du type : « **Mesure N – ADC0H = XX – ADC0L = YY – Vin en mV : yyyy** », comme représenté ci-dessous :

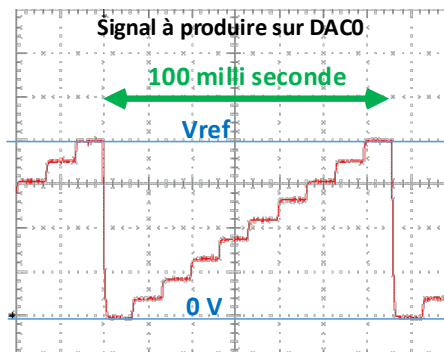
Mesure 5 -ADC0H:0x03 – ADC0L:0xf4 – Vin en mV: 300

Exemple du `printf` qui a permis l'affichage ci-dessus :

```
printf (" Mesure %5u -ADC0H:%#2.2bx – ADC0L:%#2.2bx – Vin en mV: %#2.2u \n\r", CP_Mesure, ADC0H, ADC0L, Value_mV);
```

6. Activité 3– Configuration du DAC0 – Premiers essais de conversion N/A.

- Configurer le DAC0 (convertisseur digital-analogique) pour qu'il soit en mesure d'exécuter des conversions par déclenchement logiciel (la conversion DAC est déclenchée par l'écriture dans le registre DAC0H).
- Cette configuration sera codée dans une fonction `DAC0_Init()` (le squelette de la fonction est donné). Dans la configuration, on fera en sorte que les données 12 bits transmises au DAC soient les 12 bits poids faibles du couple de registre DAC0H-DAC0L (calage à droite).
- La procédure de test sera la suivante : en complétant la fonction d'interruption périodique du `timer2`, on cherchera à produire une rampe de tension en marche d'escalier de 0V à `Vref` et de 100ms seconde, signal observable sur la broche de sortie DAC0 du convertisseur DAC0.
- Visualisez la rampe sur l'oscilloscope. Vérifiez bien sa durée (100 ms) et que la tension est bien comprise entre 0V et `Vref`.



7. Activité 4 – Conversion A/N numérique périodique.

On souhaite désormais réaliser des conversions A/N périodiques.

Aussi, on codera une seconde fonction de configuration de l'ADC0 nommée `ADC0_Init_Activ4()` (elle se substituera à `ADC0_Init()`), pour que désormais la conversion soit déclenchée périodiquement par un dépassement (overflow) du timer3. D'autre part, cette nouvelle configuration de l'ADC0 permettra de déclencher une interruption de fin de conversion ADC0.

1. En partant de la fonction `ADC0_Init()`, coder `ADC0_Init_Activ4()`. Cette fonction configurera l'ADC0 pour déclencher une acquisition à chaque overflow du timer3, et la conversion sera faite sur un signal appliqué sur la voie AIN0.
2. On souhaite avoir une fréquence d'échantillonnage de 10Khz. Aussi, configurer le timer3 au travers de `Timer3_Init()` (le squelette de la fonction est donné) pour qu'il produise des overflows toutes les 100µS.
3. Coder la fonction d'interruption `ISR_ADC0`. Mettre en place un drapeau matériel sur P3.4 (`VISU_INT_ADC0`) qui permettra de visualiser l'exécution du programme d'interruption ADC0 et de mesurer sa récurrence.
4. Par ailleurs, pour vérifier que l'acquisition de signal (la voie AIN0) est correcte, on enverra sur la sortie DAC0, le résultat de conversion ADC0 obtenu à chaque interruption.
5. Vérifier sur l'oscilloscope que le signal injecté sur Ain0 (fréquence 100Hz) et le signal de sortie DAC0 (on travaille avec des signaux 0-1V d'excursion) sont quasiment identiques.
6. Pour un signal d'entrée à 1Khz sur Ain0, combien a-t-on de points de conversion sur une période du signal d'entrée (observation du signal DAC0) ?
7. Pour un signal de 9KHz en entrée, qu'observe-t-on en sortie du DAC0 ? Quelle est sa fréquence ? Conclure.